

# Section 04: Operators, Booleans, and Conditionals **Concept Map**

The full path through this section - we open here before each concept, then dive in.



# Operators: Keywords Covered

Section 04: Operators, Booleans and Conditionals - the keywords in bright green are the new ones we cover here.

|                                                           |                                                                              |                                                              |
|-----------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| <div>Values</div> <div>TrueFalseNone</div>                | <div>Classes</div> <div>class</div>                                          | <div>Context Managers</div> <div>with</div>                  |
| <div>Logic &amp; Membership</div> <div>andornotinis</div> | <div>Error Handling</div> <div>tryexceptfinallyraise</div> <div>assert</div> | <div>Cleanup &amp; Flow</div> <div>passdel</div>             |
| <div>Conditionals</div> <div>ifelifelse</div>             | <div>Modules &amp; Imports</div> <div>importfromas</div>                     | <div>Generators &amp; Async</div> <div>yieldasyncawait</div> |
| <div>Loops</div> <div>forwhilebreakcontinue</div>         | <div>Scope</div> <div>globalnonlocal</div>                                   | <div>Soft Keywords</div> <div>matchcase_underscoretype</div> |
| <div>Functions</div> <div>defreturnlambda</div>           |                                                                              |                                                              |

keyword

New in this section

keyword

Covered earlier

keyword

Not covered yet

# Operators: Arithmetic Operators

**Arithmetic operators** are the symbols Python uses to do math on numbers - from add, subtract, multiply, and divide to floor division, modulo, and power.

| # | CONCEPT                                                                                                                                        | SAMPLE CODE                                   |
|---|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 1 | <b>Everyday math operators</b> <code>+</code> <code>-</code> <code>*</code> <code>/</code><br>Add, subtract, multiply, and divide two numbers. | <code>print(a + b) # 22: addition</code>      |
| 2 | <b>Three more operators</b> <code>//</code> <code>%</code> <code>**</code><br>The whole-number quotient, the remainder, and powers.            | <code>print(a // b) # 3: quotient</code>      |
| 3 | <b>Number helpers</b> <code>abs()</code> <code>divmod()</code><br>Size without a sign; quotient and remainder together.                        | <code>print(divmod(17, 5)) # (3, 2)</code>    |
| 4 | <b>Augmented assignment</b> <code>+=</code> <code>-=</code> <code>*=</code><br>Update a variable using its own value, in shorthand.            | <code>score += 50 # score = score + 50</code> |

# Operators: Comparison Operators

A comparison operator tests how two values relate and answers with a Boolean - **True** or **False**.

| # | CONCEPT                                                                                                                                         | SAMPLE CODE                      |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 1 | <b>Equal and not equal</b> <code>==</code> <code>!=</code><br>Check if two values are equal or not equal.                                       | <pre>print(age == 20)</pre>      |
| 2 | <b>Ordering comparisons</b> <code>&gt;</code> <code>&lt;</code> <code>&gt;=</code> <code>&lt;=</code><br>Ask which value is greater or smaller. | <pre>print(age &gt; 18)</pre>    |
| 3 | <b>Compare with None</b> <code>is</code> <code>is not</code><br>Test for None with is, never with ==.                                           | <pre>print(result is None)</pre> |

# Operators: Boolean Logic (1 of 2)

**Boolean logic** combines True and False values with operators like and, or, and not so a program can make decisions.

| # | CONCEPT                                                                                         | SAMPLE CODE                                  |
|---|-------------------------------------------------------------------------------------------------|----------------------------------------------|
| 1 | <b>Logical AND</b> <code>and</code><br>True only when both sides are true.                      | <pre>print(is_member and age &gt;= 18)</pre> |
| 2 | <b>Logical OR</b> <code>or</code><br>True when at least one side is true.                       | <pre>print(is_member or age &lt; 18)</pre>   |
| 3 | <b>Logical NOT</b> <code>not</code><br>Flips a True/False value to its opposite.                | <pre>print(not is_member)</pre>              |
| 4 | <b>Membership test</b> <code>in</code><br>Tests if a value sits inside text or a collection.    | <pre>print("a" in "Kalyan")</pre>            |
| 5 | <b>Truthy and falsy</b> <code>bool()</code><br>Empty values are false; everything else is true. | <pre>print(bool(0), bool(""))</pre>          |

# Operators: Boolean Logic (2 of 2)

**Boolean logic** combines True and False values with operators like and, or, and not so a program can make decisions.

| # | CONCEPT                                                                                                                                       | SAMPLE CODE                               |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| 6 | <b>Every / any item</b> <code>all()</code> <code>any()</code><br><code>all()</code> needs every item true; <code>any()</code> needs just one. | <pre>print(all(checks))</pre>             |
| 7 | <b>Negated membership</b> <code>not in</code><br>True when a value is NOT inside.                                                             | <pre>print("z" not in "Kalyan")</pre>     |
| 8 | <b>Chained comparison</b><br>Test a whole range in one line, like a score 0 to 100.                                                           | <pre>print(0 &lt;= score &lt;= 100)</pre> |
| 9 | <b>Operator precedence</b><br>Math first, then comparisons, then and / or.                                                                    | <pre>print(2 + 3 == 5)</pre>              |

# Operators: Truthy and Falsy

Empty things count as False; anything with content counts as True. This is how if treats any value.

## Falsy

`0`, `""`, `[]`, `None` act as False.

## Truthy

Any number (not 0) or non-empty text acts as True.

## Why it helps

`if name:` means "if name is not empty".

## TRUTHY / FALSY

```
bool(0)      # False
```

```
bool("")     # False
```

```
bool(5)      # True
```

```
bool("hi")   # True
```

---

Watch out: the text '0' is truthy - only the number 0 is falsy.

# Operators: **if** / **elif** / **else**

**if** / **elif** / **else** checks conditions from top to bottom and runs the block under the first one that is True; **else** handles every remaining case.

| # | CONCEPT                                                                                      | SAMPLE CODE                                             |
|---|----------------------------------------------------------------------------------------------|---------------------------------------------------------|
| 1 | <b>if / elif / else chain</b><br>Turn a student's marks into a letter grade.                 | <pre>if marks &gt;= 90:</pre>                           |
| 2 | <b>Ternary one-liner</b> <code>if..else</code><br>Choose one of two values in a single line. | <pre>label = "adult" if age &gt;= 18 else "minor"</pre> |
| 3 | <b>Nested if</b><br>Check an inner condition only after the outer one is True.               | <pre>if is_admin:</pre>                                 |

# Conditionals: One-Line **if/else**

When you just pick between two values, write it on one line: `value_if_true` if condition else `value_if_false`.

## Compact

A whole if/else choice on a single line.

## Reads left to right

**A if test else B.**

## Good for

Picking one of two values to store or print.

## TERNARY

```
age = 20
tag = "adult" if age >= 18 else "kid"
print(tag)    # adult
```

---

Great for a simple pick - use a real if when it gets complex.